

Reviewer Pack — Minimal Rank of Formal Power Series Bounds Tseitin Circuit S...

Entry 9b165d0f2a6e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 18:07:38 UTC

Minimal Rank of Formal Power Series Bounds Tseitin Circuit Size

Entry ID: 9b165d0f2a6e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 18:07:38 UTC

1. Conjecture

Field A (mathematical branch): Formal Power Series Theory **Field B** (complexity object): Complexity Theory: Tseitin Circuit Complexity

Statement:

[‘Let F be a formal power series over the field with two elements $F(x) = \sum_{r=0}^{\infty} a_r x^r$, where a_r are in $\{0,1\}$. Define $R(F)$ as the minimal number of non-zero coefficients in any monomial of F . For any 3-CNF formula φ , there exists an associated formal power series $G(\varphi)$ such that $R(G(\varphi)) = \Theta(\log n)$, where n is the number of variables in φ .’, ‘For all instances with property P, property Q holds.’, ‘Property P refers to the presence of a unique solution in the 3-CNF formula φ , and property Q states that the Tseitin circuit size for φ is bounded by a function of $\log n$.’]

Rationale (proposer’s reasoning):

[‘Formal power series can encode combinatorial structures in an algebraic manner. By examining the minimal rank of formal power series associated with 3-CNF formulas, we may uncover non-trivial connections between algebraic properties and circuit complexity.’, ‘This bridge between Formal Power Series Theory and Tseitin Circuit Complexity could reveal new insights into the structure of Boolean functions and their complexity.’]

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 58f156100300a55d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given 3-CNF formula φ with n variables, if $R(G(\varphi))$ is within $\Theta(\log n)$ of $\log n$ and the Tseitin circuit size for φ is below a function of $\log n$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "formal power series" AND "Tseitin circuit complexity" - "minimal rank of formal power series" AND "circuit size" - "unique solution in 3-CNF" AND Tseitin circuit

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            A[j][i:] = [A[j][k] - factor * A[i][k] for k in range(i, n)]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([8, 10, 15, 20, 30, 40])
    instances_tested = 30
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for _ in range(instances_tested):
        # Generate a random 3-CNF formula with n variables
```

```

clauses = []
for _ in range(n * (n - 1) // 2):
    literals = [random.choice([f"x{i}", f"~x{i}"]) for i in range(1, n+1)]
    clause = random.sample(literals, 3)
    clauses.append(" & ".join(clause))
formula = " | ".join(clauses)

# Compute the associated formal power series G( $\varphi$ )
F = [0] * (n + 1)
for i in range(n):
    F[i] += random.choice([0, 1])
F[n] = 1

R_F = sum(1 for a_r in F if a_r != 0)

# Calculate the Tseitin circuit size for  $\varphi$ 
Tseitin_size = n + len(clauses) * 2 + 1

# Check the conjecture
if abs(R_F - math.log(n, 2)) > 1 and Tseitin_size > 2 * math.log(n, 2):
    conjecture_holds = False
    counterexample = f"Formula: {formula}, R(F): {R_F}, Tseitin size: {Tseitin_size}"
    break

total_metric_value += abs(R_F - math.log(n, 2))

return {
    "metric_name": "R(F) - log(n)",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_"))
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.6f} std={std_metric_value:.6f} support_")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.6f} std={std_metric_value:.6f} support_")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

& ~x6 & ~x12 | ~x6 & x1 & ~x11 | x7 & x11 & x15 | ~x11 & x3 & ~x15 | ~x13 & ~x11 & ~x10 | ~x11 & ~x15 & x1 |
TRIAL: {"seed": 37, "metric_name": "R(F) - log(n)", "metric_value": 0.000000, "instances_tested": 30,

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify if the conjecture's support conditions are met. | next: Re-run the test to ensure it completes successfully and provides the necessary data to evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14054
2	propose	ollama_remote	glm4:latest	0	0	11376
3	preregistration	ollama_remote	glm4:latest	0	0	5758
4	novelty	ollama_remote	glm4:latest	0	0	4680
5	novelty	ollama_remote	glm4:latest	0	0	5878
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11434
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10656
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9219
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12425
10	judge	ollama_remote	glm4:latest	0	0	8435

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93915 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/9b165d0f2a6e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9b165d0f2a6e.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9b165d0f2a6e.tar.gz (if generated)